Persistence, offline profit, and prestige

Table of contents

Plan stable save identifiers

The default save format stores holder data and identifies referenced ScriptableObjects by asset `name`. On load, the runtime searches every Resources folder for a same-type asset with that name.

Before release:

- Put every persisted definition below a `Resources` folder.
- Make same-type names globally unique across Resources.
- Treat names as stable schema identifiers.
- Avoid duplicating an asset and leaving both copies with the same name.
- Document intentional renames and migrate existing JSON or asset identifiers before shipping.

The loader logs when no matching asset exists or when more than one match exists.

Configure SaveAndLoad

Add one `SaveAndLoad` component after the economy managers exist.

Field	Behavior
`savePath`	File-name text used by the current `Path` property
`Path`	`System.IO.Path.Combine(Application.persistentDataPath, savePath + ".json")`
`autoSaveInterval`	Seconds between automatic `Save()` calls while active
`maxOfflineSeconds`	Upper bound on elapsed time applied by `Load()`; default 86,400 seconds

Call:

```
SaveAndLoad.Instance.Load(out SaveFile restoredFile);
```

after all used managers are present. Call `Save()` at deliberate lifecycle points such as pause, application quit, transaction completion, or a debounced save-worthy event. Use the save-worthy event to queue a write after economic mutations; do not synchronously save once for every holder callback. See Coalesce autosaves with EconomyChangedEvents (`05-scripting-reference.pdf`).

The component's auto-save timer starts at zero, so its first `Update` saves immediately. Load from `Awake` / `Start` before that first update, or keep the component disabled until initialization finishes. Otherwise a new in-memory state can overwrite an existing file before it is restored.

`IrreversiblyDeleteSave()` deletes the current path and the legacy pre-`Path.Combine` location. Put a confirmation step in front of any player-facing delete button.

Saves written by earlier versions to the legacy concatenated location are migrated automatically on `Awake()` / `Load()`.

Try it: open `EasyIdleGame > Demos > FeatureDemos > CustomSaveAndAutosave > CustomSaveAndAutosave.unity`, then select the adjacent `ReadMe.asset` for setup notes and its resource prefix.

What the default SaveFile stores

The default `SaveFile` includes:

- Currency and business holders.
- Manager, location, shop item, upgrade, and achievement holders when their managers exist.
- Player level, lifetime currency/business/boost tracking, and custom stats.
- Boost inventory and active boost timers.
- Prestige count and scaling data.
- Daily reward initialization date and claimed days.
- Save timestamp.

Timed upgrade queues are serialized inside their `UpgradeHolder`. Stock and production state are stored inside business holders.

Managers that do not exist at save time contribute no data. Create optional managers before loading data that expects them.

Load and apply offline progress

`SaveAndLoad.Load()` performs three actions:

1. Deserialize JSON and call `SaveFile.Load()`.
2. Calculate `DateTime.Now - file.saveTime`.
3. Clamp that duration to `maxOfflineSeconds`, calculate profit, and apply it when the duration is positive.

Do not call `ProfitCalculator.CalculateAndApplyProfit` again for the same elapsed period.

To show a welcome-back summary, subscribe to `ProfitCalculator.OnProfitApplied` before load or inspect the emitted `ProfitApplicationSummary` from your own load flow.

`ProfitApplicationSummary` separates the requested `ProfitData` from what caps actually allowed:

- `AppliedCurrencies`
- `AppliedBusinesses`
- `AppliedProductionTimes`
- `AppliedStockpiles`

Use these applied values in player-facing UI when maximum amounts or group capacity may clamp output. Calculated and applied amounts are not interchangeable: wallet capacity, group capacity, and other constraints can make the applied result smaller than the calculated profit, so do not promise the raw calculated amount in a final grant screen. Time skip uses the same simulation and should also report the applied result.

Use ProfitData without applying it

For a preview:

```
TimeSpan elapsed = TimeSpan.FromHours(2);
ProfitData preview = ProfitCalculator.CalculateProfit(elapsed);

BigNumber coins = preview.GetCurrencyAmount(coin);
List<Output> stockpile = preview.GetBusinessStockpile(factory);
```

`CalculateProfit` works on cloned holder state and restores active boosts after the calculation. It does not apply currency or business changes until `ApplyProfit` is called.

Apply once:

```
ProfitApplicationSummary summary = ProfitCalculator.ApplyProfit(preview);
```

or combine both operations:

```
ProfitApplicationSummary summary =
    ProfitCalculator.CalculateAndApplyProfit(elapsed);
```

Understand offline simulation

The calculator divides elapsed time into at most 30 segments and simulates production chains, inputs, outputs, speed, boosts, managers, caps, and stockpiles over those segments.

It supports:

- Businesses that output other businesses.
- Currency or business production inputs.
- Consumable producers.
- Production and speed multipliers.
- Active boost and manager timer simulation.
- Auto-produce override boosts.
- Business/currency maximums and typed-group capacities.
- Stockpiles for non-auto-collected businesses.
- Contextual Offline upgrade filters.
- Registered production output modifiers.

Important implications:

- It is a bounded simulation, not an unbounded frame-by-frame replay.
- Random production is represented through average/optimistic output paths appropriate to the offline calculation.
- Passing a non-positive duration is not a useful calculation; validate elapsed time before calling the API directly.
- `TimeSkipBoost` uses the same calculation path and may optionally advance active boost timers.
- The default load clamp is separate from the `idleProductionTimeCap` override type; if your game uses both, define one policy and apply the effective cap consistently in custom load code.
- Active boost and manager timers advance once per simulation segment, regardless of the number of tracked businesses.

Configure prestige

Add `PrestigeManager`, then configure:

1. `prestigeRequirements` with normal `Lock` entries.
2. `prestigeCostMultiplier` for requirement growth.
3. `multiplyLevelOnPrestige` if level/prestige-count requirements should also scale.
4. `maxPrestigeCount` (`0` means infinite).
5. Persistent `prestigeUpgrades` and their multiplier.
6. Immediate `prestigeRewardTables` and optional reward scaling by the new prestige count.
7. Reset flags and exclusion lists.

Call:

```
if (PrestigeManager.Instance.TryPrestigeGame(
        out List<Output> rewards,
```

```
        out ResetSummary resetSummary))
{
    // Present the actual rewards and what was reset.
}
```

The operation validates requirements, resets configured systems, increments prestige state, updates scaled requirements, grants rewards with `SourceType.Reset`, restores location starters where needed, raises `OnPrestige`, and marks state save-worthy.

Preview the confirmation UI with `GetExpectedPrestigeRewards`; never roll the reward tables just to render a preview. Handle a failed `CanPrestige` or operation result without clearing project state.

Control reset scope

Prestige has independent flags for:

- Manager holders and manager unlock state.
- Business holders and business unlock state.
- Active boosts, boost inventory, and boost unlock state.
- Currency holders.
- Player level.
- Upgrade holders and upgrade unlock state.
- Location holders and location unlock state.
- Achievement holders and achievement unlock state.
- Shop item unlock state. Shop ownership is not reset by the current prestige manager.

Every resettable definition type supports explicit exclusions. Most also support typed-group exclusions. Currencies use the serialized field name `exludedCurrencies` for compatibility; keep the misspelling when referring to the public field in code.

Unlock-state flags are independent of holder reset flags. Preserve a holder but reset its `Unlock Only Once` state only when that behavior is intentional.

Add per-call prestige exclusions

Use `PrestigeResetOptions` for a one-time preservation policy without mutating the Inspector configuration:

```
var options = new PrestigeResetOptions
{
    additionalExcludedCurrencies = new[] { eventCurrency },
    additionalExcludedUpgradeGroups = new[] { permanentUpgradeGroup },
    notifySaveWorthyStateChanged = true
};

PrestigeManager.Instance.TryPrestigeGame(
    out List<Output> rewards,
    out ResetSummary summary,
    options);
```

Per-call exclusions are combined with configured explicit and group exclusions.

Use ResetSummary for downstream cleanup

Let the package reset the economic holders, then use `OnPrestige` for project-owned cleanup: physical boards, scene objects, or travel back to a default world. `ResetSummary` reports which systems were configured to reset and the final preserved item lists; read it when downstream systems need to know exactly what was reset or preserved, and use it as the audit record for reset confirmation UI and tests.

Preserve location starter state

Locations can grant starter output tables on first unlock. After prestige, `LocationsManager.RestoreStarterOutputs()` reapplies starter outputs for preserved/unlocked locations and restores the default location when appropriate.

Design the exclusion set together with starter rewards:

- A preserved location may need its starter business/currency outputs restored after those other systems reset.
- A reset location should not remain active.
- The default location must be unlockable or free enough to become active after a full reset.

The included prestige tests cover preserved locations, default location rewards, and independent per-system unlock-state flags.

Test the lifecycle

Before release, test this sequence with a copy of real progression data:

1. Create current, lifetime, and bought amounts through both purchases and free grants.
2. Start active production, create a stockpile, activate boosts/managers, and queue a timed upgrade.
3. Unlock a location, claim daily rewards, and progress achievements.
4. Save and inspect `SaveAndLoad.Path`.
5. Restart the scene/application and load once.
6. Confirm offline progress respects costs, caps, chains, stockpiles, and timers.
7. Prestige with explicit and group exclusions.
8. Save again, restart, and confirm the post-reset state is stable.
9. Repeat with an old save fixture to validate migration seeding.